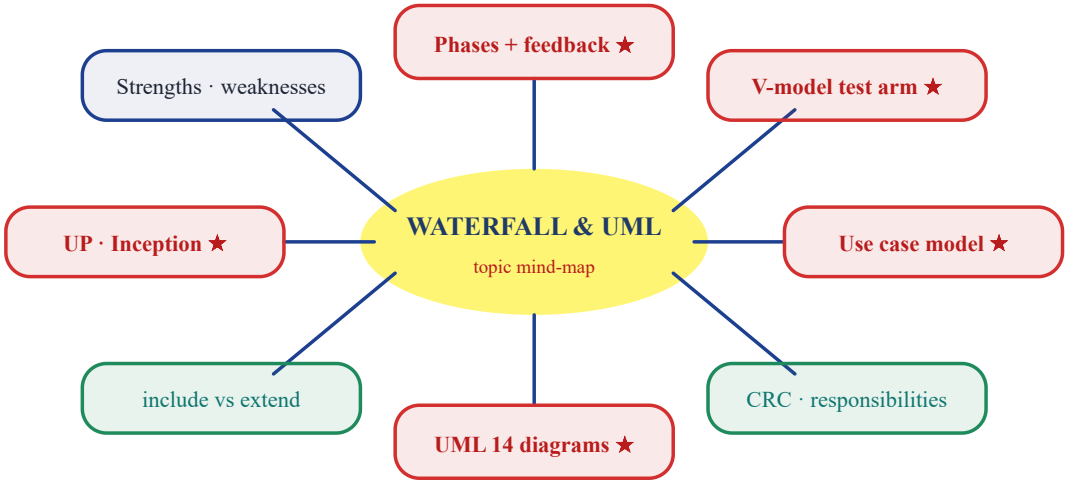
Waterfall, Inception & UML

Phases · V-model · Inception · use cases · UML · OO design: exam-ready in 11 sheets



↑ the whole topic on one page: red = high-yield, green = method / applied

Syllabus coverage

"…Sequential **waterfall life cycles, Inception**. **Use case model** … **UML diagrams** … **Designing Objects with responsibilities**."

High-yield list

① Waterfall: linear, rigid
② Feedback comes late
③ V-model = the test arm
④ Inception ≈ 10% effort
⑤ include mandatory
⑥ filled ◆ = composition

Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

Contents

✓ Waterfall model & phases …… p. 2
✓ Strengths, weaknesses, V-model …… p. 3
✓ Unified Process & Inception …… p. 4
✓ Inception vs waterfall reqs …… p. 5
✓ The use case model …… p. 6
✓ include/extend, SSD, sequence …… p. 7
✓ UML: 14 diagrams & relations …… p. 8
✓ OO design & CRC cards …… p. 9
✓ Night-before sheet …… p. 10
✓ Practice MCQs + written …… p. 11

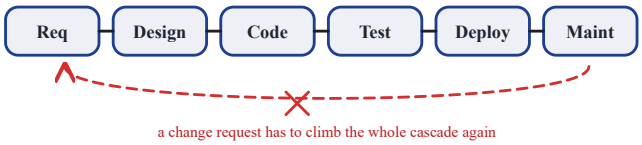
1 The Waterfall (Linear Sequential) Model

Definition: Development as a **cascade of phases**: each phase is **completed, reviewed and signed off** before the next starts, and the deliverable of one phase is the only input of the next. Phases do not overlap and a finished phase is not re-opened.

> output of n =
> only input of n+1

#	Phase	Main activity	Deliverable
1	**Requirements analysis & specification**	Elicitation, analysis, SRS, feasibility	SRS (user + system)
2	**System & software design**	Architecture, data, module, UI design	system + software design docs
3	**Implementation (coding)**	Coding, code review, unit testing	source code, build
4	**Integration & system testing**	Integration, functional testing, fixing	test plan, report, defect log
5	**Deployment**	Installation, migration, training	released system, manual
6	**Maintenance**	Correction, adaptation, improvement	patches, updated docs

- **Linear and rigid** — one active phase, no overlap
- **Documentation-heavy** — every phase ends in a reviewed deliverable
- **Requirement freeze** — later changes go through change control
- **Verification is late** — feedback sits in the **testing phase**



six phases, one direction only

Memory trick: **R-D-C-T-D-M**: **R**equirements → **D**esign → **C**ode → **T**est → **D**eploy → **M**aintenance. You may only go **down** the falls.

Exam trap: Royce's 1970 paper drew **verification arrows going back** to earlier stages — the feedback-free waterfall is a textbook simplification.

IT link: A public-sector e-tender (tender → SRS → fixed-price contract → milestones) is classically waterfall: the buyer must own a frozen scope document before money is committed.

2 Strengths, Weaknesses & the Feedback Problem

Advantages:

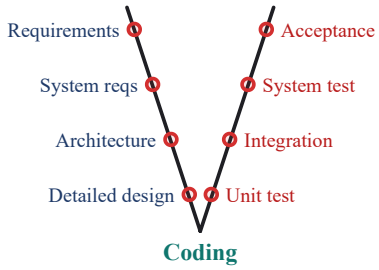
- Simple to plan and **manage**
- Clear **milestones and deliverables** → progress measurable
- Each phase staffed by **specialists**
- Thorough **documentation** — handover, audit, maintenance
- Fits **small projects with stable requirements** (BPO, fixed-price tender)

Disadvantages:

- No overlap → **long elapsed time**, late working software
- Changes are **costly**: re-entering a phase voids every later document
- Requirement errors surface only in **testing**
- The user sees **nothing running** until the end
- Poor fit for **large, novel or fast-changing** systems

The feedback problem: Waterfall assumes the requirements can be **fully and correctly stated before any code exists**. But a user can only judge software by **running** it, so the first real feedback arrives after most of the budget is spent — and a late change means redoing design, code and tests.

> cost of fixing a defect: req 1× · design 5× · code 10× · test 15–50× · production 100–1000×



each development level is paired with the test that proves it

Where it still fits	Where it fails
small project · stable requirements	ambiguous or volatile requirements
proved technology · documented work	large, distributed, long-lived systems
fixed-price milestone contract	innovation risk · fast market · heavy UI work

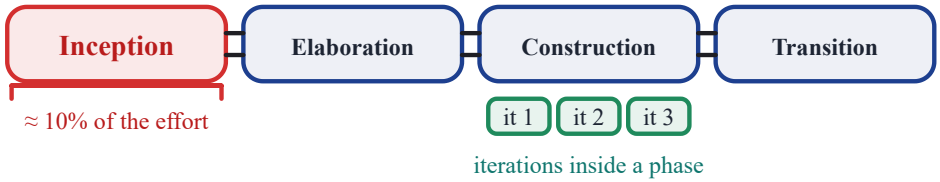
The V-model ✍️: Put the same sequential development stages on the **descending left arm** and the matching **test level** on the **ascending right arm**. It fixes **where** testing is planned, not **when** feedback arrives. **Verification** = are we building the product *right*; **validation** = are we building the *right* product. **Testing** finds that errors exist; **debugging** corrects them.

Exam trap: The V-model does **not** mean "test early every iteration" — it is still sequential; and the counterpart of **requirements is acceptance testing**.

3 The Unified Process and Its Inception Phase

Unified Process (UP): A **use-case-driven, architecture-centric, iterative and incremental** process **framework** created at Rational by Booch, Rumbaugh and Jacobson with Cockburn. Each project **tailors** it; the Rational Unified Process (RUP) is Rational's packaged version.

Phase	What happens in it	Milestone
Inception	business case, vision, initial scope, rough estimate, first risk list	the **Vision** — "is it worth doing?"
Elaboration	most requirements, the domain model, the architecture prototype	the **Architecture baseline**
Construction	build the rest, refine the design, test continuously	working **increments**
Transition	deployment, migration, training, support handover	the **released product**



a phase is a long-term goal · an iteration is a short cycle

four phases, each repeated in iterations; the milestone of a phase answers one question

Inception: The first and shortest phase. Its only question is **"should this project start at all?"** It fixes the **vision**, **business case**, **initial scope**, a **rough estimate** and an **initial risk list**.

1. Decide the **scope and size**
2. Document the **business case**
3. Rough-estimate cost and duration
4. Form the **initial risk list**
5. Identify the **stakeholders**
6. Define the **vision** and high-level use cases

Inception activity	Produces
Stakeholder identification	users, clients, sponsors
Requirements elicitation	high-level needs only
Feasibility study	technical, operational, economic
Initial risk assessment	risk list, impact, mitigation
High-level use cases	~10–20 cases, no detail
Project estimation	rough size, cost, schedule

Outputs of inception: Vision & business case · initial risk list · initial high-level use-case model · rough size, cost and schedule. **Effort guideline: about 10% of the project effort**; the estimate is **rough**, typically **±30%**.

4 Inception vs the Waterfall Requirements Phase

Aspect	Waterfall requirements	UP Inception
Scope covered	complete, detailed, **frozen**	high-level only, **incomplete by design**
Main output	SRS	**vision**, business case, initial use cases
Timing	one long phase before any design	short first phase; elaboration continues it
Accuracy	claims to be exact	order of magnitude, **±30%**
Change	change request + impact analysis	normal — the use-case model keeps growing
Risk	not explicitly addressed	an **initial risk list** is a first-class output
Testing	later, via the V-model	continuous, risk-driven, per iteration

Feasibility: five questions

- **Technical** — can it be built with the available technology and skills?

Initial risk assessment

A **risk** is an uncertain future event with a probability and an impact; **exposure = probability × impact**. The

- **Economic** — does the benefit beat the cost? (cost-benefit, payback, NPV)
- **Operational** — will people use it, and can the organisation run it?
- **Legal** — data-protection and e-government compliance
- **Schedule** — deliverable in the required time?

The study ends in a **go / no-go** decision — cancelling a weak project is a legitimate result.

chain is **identify → assess → mitigate → monitor**, with mitigation by avoid, reduce, transfer or accept. Inception produces only the **first** risk list; full risk analysis and COCOMO are in the Project Management notes.

> **Memory trick:** Inception = **"SCOOPE"** — **S**takeholders, feasibility, **C**ost/effort estimate, **O**utput = vision & use cases, **P**roject go/no-go, **E**licit only high-level needs.

> **Exam trap:** Do **not** write "Inception replaces the requirements phase" — it only *frames* requirements; the bulk are elaborated later. And do not call UP "waterfall renamed": UP is **iterative**, each phase holding many iterations, while waterfall is strictly sequential.

> **IT link:** A real project does exactly this: a one- or two-week spike produces a vision note, a handful of use cases and a risk list — then the funding decision is taken.

> **Rough estimation inside inception ✍️:** Size the work by **function points**, **use-case points** (weight each use case by complexity), **analogy** or **expert judgement**; then effort × rate ≈ cost. Inception needs only an **order of magnitude** — the firm figure arrives after elaboration. COCOMO and the full risk matrix are in the Project Management notes.

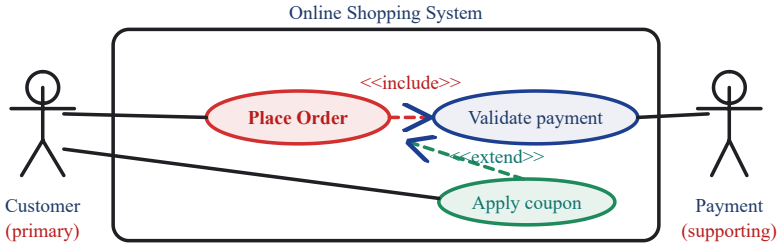
5 The Use Case Model for Requirement Writing

> **Definition:** A **use case** is a sequence of transactions that yields a **meaningful result with a particular actor** (Ivar Jacobson). It abstracts **behaviour** in the user's language — not a design, a screen or a test case.

Element	What it states	"Place Order" example
Actor	a **role** played by a person or external system, outside the boundary, drawn as a stick figure	Customer; Payment Gateway
Precondition	what must already be true — an **assumption**, never the trigger	registered, logged in, cart not empty
Main flow	the basic sequence that always delivers the goal	checkout → summary → pay → confirm
Alternative flows	variations, error and exception paths	card declined; stock changed; cancel
Postcondition	what is guaranteed true after success — a **result**	order confirmed, stock reserved
Trigger	the boundary event that starts the use case	customer clicks "Checkout"

- **Primary actor** — whose goal the use case delivers; it drives the system
- **Supporting / secondary actor** — provides a service the use case needs (Payment Gateway); a **tertiary** actor only benefits indirectly (Courier)
- An actor is a **role, not a person**: **one person may play several actors**, one actor several people.

Worked example: "Place Order" ✍️: **Main flow**: ① customer selects Checkout → ② the system shows the summary → ③ the customer enters card details → ④ the gateway authorises → ⑤ the system creates the order, reserves stock and confirms → ⑥ the order ID is shown. **Alternative flows**: card declined → another card; stock changed → recalculate; cancel → no order.



a use case diagram: actors outside the boundary, ovals inside it

Exam trap: A use case is **not** a screen, a class or a test case. Name it **verb + noun** at the **user-goal level**.

Memory trick: **P**recondition in, **M**ain flow through, **P**ostcondition out. An actor is a **role**, outside the box.

6 Use Case Relations, SSD & Sequence Diagrams

Relationship	When it is used	Direction & notation
<<include>>	**mandatory** — the base use case **always** performs the included one	base → included; dashed arrow, open head
<<extend>>	**optional** — added only under a condition; never completes alone	extending → extended
Generalisation	a specialisation inherits the flow and adds or overrides steps	specialised → general; hollow triangle

System Sequence Diagram (SSD): Draws the **whole system as one black box**: only events crossing the boundary appear. Rules (Cockburn): **one event per step**, names in **verb + noun** form, **no internal messages**, no loops.

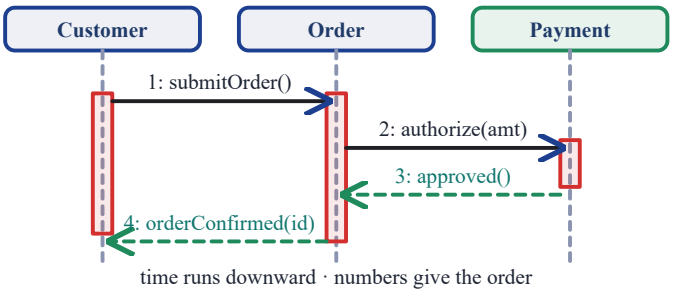
SSD for "Place Order" ✍️:

1. Customer → System: **createOrder(items)**
2. System → Customer: **paymentRequired(total)**
3. Customer → System: **makePayment(cardDetails)**
4. System → Customer: **orderConfirmed(orderId)**

A gateway call is **inside** the box here — that is why an SSD is not a sequence diagram.

Reading a sequence diagram

- **Time flows downward**; read each message **left → right**, then move down.
- **Lifelines** = the objects taking part.
- **Numbered messages** give the order.
- **Activation bar** = focus of control.
- **Filled arrow** = synchronous · **open arrow** = asynchronous · **dashed** = reply · **[guard]** = condition.
- **Dot** = exists · **open circle** = created · **X** = destroyed.



the same information the SSD hides, now opened up

Communication (= collaboration) diagram: The **same interaction** redrawn as a graph of **object links**, messages numbered on the links: it shows **who is connected to whom** (structure), not when things happen (time).

Exam trap: "Withdraw Cash **always** performs Authenticate User" → **<<include>>**. "A discount **only for members**" → **<<extend>>**. Same arrow — the **direction and the condition** tell them apart.

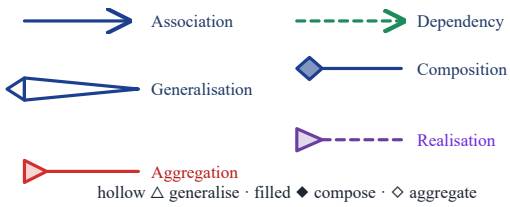
7 UML: What Each Diagram Is For

UML: The OMG standard visual language for specifying, visualising and documenting software, originating at Rational in the late 1980s (Booch, Rumbaugh, Jacobson) and an OMG standard since 1997. UML 2 has **14 diagrams**: **structural** (what exists) and **behavioural** (what happens).

Structural (7)	What it is FOR
Class	static structure: classes, attributes, operations
Object	a snapshot of *instances* and their links
Composite Structure	how a classifier is built from parts
Component	modules, interfaces, dependencies
Deployment	artifacts on hardware nodes and links
Package	grouping into namespaces / folders
Profile	stereotypes tailoring UML to a domain

Behavioural (7)	What it is FOR
Use Case	actors and the goals they want from us
Activity	workflow: flow, decisions, swimlanes
State Machine	life of one object: states, events
Sequence	interaction over **time**: lifelines + messages
Communication	the same interaction as an object graph
Interaction Overview	how interaction diagrams fit together
Timing	object values changing over time

Relationship	Meaning → notation
Association	"a knows b"; role, navigability, multiplicity → **solid line**
Aggregation	weak whole–part, parts live alone → **hollow diamond**
Composition	strong whole–part, one owner → **filled diamond**
Dependency	weakest, temporary "uses a" → **dashed + open head**
Generalisation	"is-a": subclass → superclass → solid, **hollow** △
Realisation	"implements" a contract → dashed, **hollow** △



the six notations you must be able to draw

Exam trap: Filled diamond = **composition**, hollow diamond = **aggregation**; a hollow **triangle** is generalisation, not a diamond. "Collaboration diagram" is the old UML 1.x name for the **Communication diagram**.

8 Designing Objects: Principles, Process & CRC

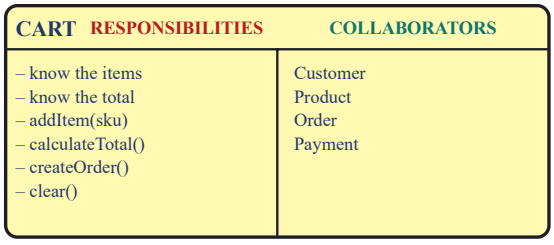
OO principle	Exactly what it means
Encapsulation (information hiding)	data and its operations bundled in one class; state private, changed only through the public interface
Abstraction	exposing **what** the object does, not **how**
Inheritance	a subclass inherits and specialises state and behaviour ("is-a")
Polymorphism	the **same message**, different behaviour, decided at **run time**

The OO design process

1. Develop or reuse a **design model**
2. Find the **design classes** for each element
3. Assign each class its **responsibilities** (CRC cards)
4. Assign each class its **collaborators**
5. Design collaborations, associations, generalisations
6. Refine classes; repeat until all behaviour is held

Responsibility = what a class **knows** (attributes, linked objects) or **does** (operations); a **collaborator** is a class that helps fulfil it.

Class	Knows (state)	Does (operations)
Customer	name, email, cart	addToCart(), checkout()
Cart	products, quantities	calculateTotal(), createOrder()
Order	orderId, lines	confirm(), generateInvoice()
Payment	method, amount	processTransaction()



a CRC card (Beck & Cunningham, OOPSLA 1989)

Concept vs design class: A **domain (concept) class** is pure information — **no operations that calculate** — named as a **singular common noun**. A **design class** is any class of the design model — implementation-level, with attributes and operations in full detail; it need not be abstract.

Reuse & refactoring: **Reuse** = patterns, inheritance, composition. **Refactoring** = a **behaviour-preserving** restructuring of code, in small steps, never mixed with feature work.

Categories of object: **Entity** = the data of the use case · **Boundary** = what the user touches · **Control** = the sequencing logic.

Domain model ✍️: The **domain model** is the **conceptual** representation of the domain's real-world concepts: every domain noun becomes a **concept class** with its **attributes** and **associations**, but **no operations** — nothing calculates, nothing is implemented. Online shopping: **Customer, Order, OrderLine, Product, Payment**, data only.

Multiplicity ✍️: The number at each end of an association says **how many**: **1** exactly one · **0..1** zero or one · ***** or **1..*** many. Read it **at the far end** — **Customer 1 — * Order**: one Customer places many Orders, so read back each Order is for exactly **1** Customer. Likewise **Order 1 — * OrderLine**, **OrderLine * — 1 Product**.

Night-Before-the-Exam Sheet

Fact sheet:

Phases: Req → Design → Code → Test → Deploy → Maint	Royce, 1970 · linear, rigid, no overlap
Verification = the product the right way	Validation = the right product
V-model: reqs ↔ acceptance · arch ↔ integration	detailed design ↔ unit test
Testing finds errors · debugging removes them	UP = use-case driven, architecture centric
UP: Inception · Elaboration · Construction · Transition	each phase holds many iterations
Inception = vision, business case, scope, estimate, risks	Inception ≈ 10% of effort · estimate ±30%
<<include>> = always (base → included)	<<extend>> = only if (extending → extended)
SSD = black box, verb+noun, no internals	Sequence: time ↓, lifelines, activations
Aggregation = hollow ◆ · Composition = filled ◆	Dependency = dashed + open arrowhead
Generalisation / realisation = hollow △	UML 2 = 14 diagrams = 7 + 7
Collaboration = Communication diagram	CRC: Beck & Cunningham, 1989
Responsibility = knowing + doing	Refactoring = behaviour-preserving
Concept class = information only	Objects: entity · boundary · control

Top 14 one-liners for MCQs

1. Waterfall was proposed by W. W. Royce, 1970
2. Waterfall = linear-sequential, phases do not overlap
3. Inception is the first UP phase
8. Collaboration diagram was renamed Communication
9. Filled diamond = composition
10. Hollow diamond = aggregation

4. Inception answers "is it worth starting?"
5. Inception outputs a vision, not an SRS
6. An actor stands outside the system boundary
7. Sequence diagram = lifelines + messages in time order
11. Hollow triangle = generalisation
12. Dashed arrow, open head = dependency
13. Elaboration builds the architecture baseline
14. CRC cards were invented by Beck & Cunningham

Top traps: <<include>> vs <<extend>>: "always" vs "only if" · composition vs aggregation: filled vs hollow diamond · verification vs validation: the product vs the process · SSD vs sequence: black box vs opened up · "waterfall has no feedback at all" is wrong · Inception does not write the SRS · a use case is not a screen.

All tricks together: R-D-C-T-D-M staircase · "only down the falls" · V = development left, test right · Inception = SCOOPE · precondition in, main flow through, postcondition out · "always" = include, "only if" = extend · solid = knows, dashed = merely uses · hollow diamond is weak, filled is strong · concept classes store, design classes behave.



include and extend — same arrow, opposite job

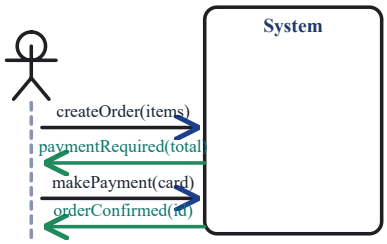
✓ Practice MCQs (exam pattern, 10 marks)

1. Which diagram uses lifelines and messages?
   (a) Class diagram (b) Sequence diagram
   (c) Deployment (d) Use case
2. A person or system interacting with it is:
   (a) Class (b) Node
   (c) Actor (d) Component
3. Which UP phase sets the business case?
   (a) Inception (b) Elaboration
   (c) Construction (d) Transition
4. A base use case **always** performs another:
   (a) <<extend>> (b) Generalisation
   (c) Association (d) <<include>>
5. Composition is drawn as which symbol?
   (a) Hollow diamond (b) Filled diamond
   (c) Hollow triangle (d) Dashed arrow
6. Aggregation is drawn as which symbol?
   (a) Hollow diamond (b) Filled diamond
   (c) Hollow triangle (d) Dotted line
7. A "depends on" relation is drawn as:
   (a) Solid arrow (b) Filled diamond
   (c) Dashed arrow (d) Solid triangle
8. Runs only when a coupon code is present:
   (a) <<include>> (b) <<extend>>
   (c) Generalisation (d) Realisation
9. "SavingsAccount is-a Account" is shown by:
   (a) Filled diamond (b) Hollow diamond
   (c) Hollow triangle (d) Dashed arrow
10. Inception uses about what share of effort?
    (a) 25% (b) 50%
    (c) 10% (d) 75%

Answer key → 1-b · 2-c · 3-a · 4-d · 5-b · 6-a · 7-c · 8-b · 9-c · 10-c

Written question 1: Describe the Waterfall model and its phases. Give two advantages and two disadvantages. 8 marks

1. **Definition** — linear-sequential: each phase is finished, reviewed and signed off before the next 2
2. **Phases** — requirements (SRS) → design → implementation with unit testing → integration and system testing → deployment → maintenance 2
3. **Advantages** — simple to plan with clear milestones; good documentation 2
4. **Disadvantages** — changes are costly; software appears late 2



SSD = one black box

Written question 2: Draw a use case diagram for online shopping with one primary actor, one supporting actor, one <<include>> and one <<extend>>, stating each arrow's direction. 7 marks **Answer: Customer** left (primary), **Payment Gateway** right (supporting); <<include>> *Place Order* → *Validate Payment*; <<extend>> *Apply Coupon* → *Place Order*.

Before the exam ✓

- ✓ Recite the six waterfall phases
- ✓ Give two advantages and two disadvantages
- ✓ Name the four UP phases and inception outputs
- ✓ Tell include from extend by the condition
- ✓ Draw all six UML notations

Neighbours: the SDLC phases are in **SE Concepts**; estimation and COCOMO in **Project Management**; responsibility patterns in **GRASP & DCD**.